AscendNPU-IR

Optimized kernels can still contain avoidable work

Computation

Operations that leave the final result unchanged.

Data movement

Extra loads, stores or layout conversions.

Execution

Work mapped inefficiently to the target.

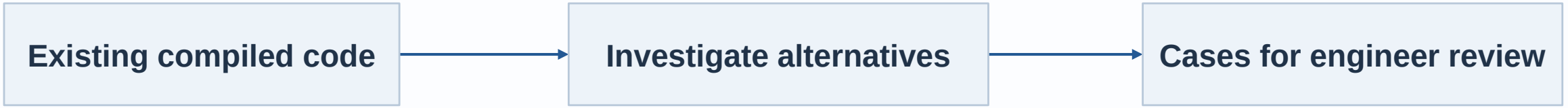
[REWORK] How optimization opportunities reach engineers today

Nightly regression tests	Catch performance regressions before they reach upstream.
Performance tickets	Trigger investigation of a slowdown in merged code.
Kernel tasks and optimization epics	Ask engineers to speed up a kernel or improve a class of optimizations.

No proactive search for missed optimizations is currently performed in the project. (To my knowledge)

Proactive optimization discovery is the gap we target

Where could an existing kernel improve even when its performance has stayed stable?



Each case takes expert time to choose a rewrite, test its behavior and interpret the comparison.

Build a framework for reproducible optimization discovery

Build an LLM-assisted framework that proposes MLIR rewrites and checks them against the original code on the same Ascend target.



It handles repeated runs and packages the evidence so compiler engineers can assess promising cases.

Five goals shape the experiment

Goal	What the experiment must establish
Find useful opportunities	Which rewrite families produce cases engineers would pursue?
Supply dialect knowledge	Which semantics, constraints and examples does the agent need?
Compare alternatives	Can one compilation route support both a check and a useful metric?
Reproduce findings	Can an engineer replay a finding and resume interrupted work?
Evaluate cost	How much expert time, model use and device time does a finding take?

A framework coordinates the investigation

A request defines the editable MLIR region, target, checks and search budget.



Versioned dialect references

Diagnostics and comparison results guide the next edit within budget.

Save the original, every candidate, the check results and resource use. Export a qualifying pair with its replay command.

Search for specific transformation families

Candidate family	Example	Basis
Arithmetic simplification	Remove a clamp made redundant by a later clamp.	LPO · LLVM
Predicates and selections	Remove a redundant NaN check before a comparison.	LPO · LLVM
Memory accesses	Combine adjacent loads when a wider load is legal.	LPO · LLVM
Vectorization	Restructure a conditional loop to enable vector instructions.	PRAGMA · kernels
Reduction organization	Change how the target executes a reduction.	PRAGMA · kernels
Layout conversions	Remove a pair of inverse conversions when legal.	Project candidate

Ascend applicability remains open. LPO §4.3 · arXiv 2508.16125 | PRAGMA §IV-D · arXiv 2511.06345

An MLIR rewrite can remove redundant work

```
Shared operands     %x : f32      %zero = 0.0 : f32      %one = 1.0 : f32
```

Original MLIR

```
%ordinary = arith.cmpf ord, %x, %x : f32
%safe = arith.select %ordinary, %x, %zero : f32
%equal = arith.cmpf oeq, %safe, %one : f32
```

Candidate MLIR

```
%equal = arith.cmpf oeq, %x, %one : f32
```

For NaN, both versions return false. Otherwise, both compare x with one.

Adapted from LPO, Figure 4(c, f). Ascend behavior remains untested.

Bound the search and the claims

Editing scope

Choose a checkpoint and operation subset. Preserve the enclosing code and restrict edits to the selected region.

Behavior

Record assumptions about aliasing, numerical precision and target behavior. Check the observable results those assumptions cover.

Evidence

Report exactly what each check covers. A runtime speedup claim requires comparable execution measurements.

Budget

Limit candidate attempts, compilation jobs, model use and NPU runs. Save partial results when a limit is reached.

A focused MVP by month 4

One supported checkpoint, one checking policy and one metric for engineer-supplied MLIR.

Month 1	Months 2–3	Month 4	Month 5	Month 6
Test a compile-and-check route on a real case.	Implement dialect context, rewriting, checks and feedback.	Replay a finding and resume an interrupted investigation.	Evaluate held-out cases and record all outcomes.	Test one refinement against the baseline.

Planned support includes one developer, part-time compiler expertise, host and model capacity, and NPU access when required.

Two uncertainties determine feasibility

Will real cases yield useful findings?

Rewrites may be invalid, trivial or already handled by a later pass. Have an engineer review each qualifying pair after replay.

What will integration take?

A saved IR fragment may need compiler state, a harness or device access. Exercise one complete route in the first month.

Also track

Time spent preparing cases and reviewing rewrites
Whether the metric tracks an improvement that matters
Which behaviors remain outside the selected checks

Judge success by useful, replayable findings

MVP target

At least one real case that an AscendNPU-IR engineer considers worth investigating and can replay.

Before the study

Fix the evaluation cases, review criteria, budgets and repeat count.

For every case

Record useful, rejected, duplicate and inconclusive outcomes, including failed replays.

Cost of a finding

Track expert effort, model use and host/device time. Compare with the current workflow to assess savings.